TaskResultsTable UI仕様書

1. 概要

1.1. コンポーネントの目的

プロジェクト内のタスク結果を一覧表示するテーブルコンポーネント。各タスクのステータス、作業終了日、レビュー完了日を表示し、詳細確認が可能です。

1.2. 使用場面

- ProgressMonitoring画面でタスク結果を確認する場合
- ProjectDetailModal内でタスク一覧を表示する場合
- タスクのステータスを一覧で確認する場合

1.3. アクセス権限

- **対象ロール**: Experiment Admin（実験管理者）
- **必須認証**: JWT認証必須
- **テナント分離**: organization_id によるマルチテナント分離

---

2. 画面構成

2.1. レイアウト構造

```
┌──────────────────────────────────────────────────────┐
| タスク名      |メソッド|ステータス|作業終了日|レビュー|
|──────────────┼────────┼─────────┼─────────┼────────|
| 材料準備      |Method1 | [実行中] |2024-01-15|   -    |
|               |        |          |          | [詳細] |
|──────────────┼────────┼─────────┼─────────┼────────|
| 材料測定      |Method1 | [完了]   |2024-01-18|24-01-20|
|               |        | [green]  |          | [詳細] |
```

```
|──────────────|────────|──────────|──────────|────────|
| データ分析     |Method2 | [待ち]    |   -      |   -    |
|               |        | [yellow] |          | [詳細] |
└──────────────────────────────────────────────────────┘
```

2.2. 主要UI要素

2.2.1. テーブル設定

項目	値
サイズ	small
ページネーション	false（親コンポーネントで制御）
rowKey	workId

2.2.2. カラム定義

カラム	幅	配置	ソート	説明
タスク名	30%	left	○	タスク名（太字）
メソッド	20%	left	○	メソッド名
ステータス	15%	center	○	ステータスTag（色分け）
作業終了日	18%	right	○	日付表示（年/月/日）
レビュー完了日	15%	right	○	日付表示（年/月/日）
操作	10%	center	-	詳細ボタン

2.2.3. ステータスの色定義

ステータス	色	表示名
pending	default（灰色）	実行待ち
in_progress	processing（青）	実行中
error	error（赤）	異常終了
review_waiting	warning（黄）	レビュー待ち
completed	success（緑）	完了

2.2.4. 詳細ボタン

項目	値
タイプ	default
サイズ	small
アイコン	EyeOutlined
テキスト	多言語対応（projectManagement.task_results.view_result）

2.3. 画面遷移

- **詳細ボタンクリック**: onViewResult(result)コールバックを実行
- 画面遷移は親コンポーネントで制御

---

3. 機能仕様

3.1. 主要機能の詳細

3.1.1. テーブル表示

処理フロー:

1. Props（results）を受け取る
2. TableコンポーネントにdataSourceとして渡す
3. 各カラムのrenderで表示形式を制御
4. ソートは自動で有効化

ソート機能:

- タスク名: 文字列の自然順ソート（localeCompare）
- メソッド: 文字列の自然順ソート
- ステータス: 優先度順ソート（pending → in_progress → error → review_waiting → completed）
- 作業終了日: 日付順ソート（nullは最後）
- レビュー完了日: 日付順ソート（nullは最後）

3.1.2. ステータス表示

ステータスラベル取得:

```
const getStatusLabel = (status: string) => {
  switch (status) {
    case 'pending': return '実行待ち';
    case 'in_progress': return '実行中';
    case 'error': return '異常終了';
    case 'review_waiting': return 'レビュー待ち';
    case 'completed': return '完了';
    default: return status;
  }
};
```

ステータス色取得:

```
const getStatusColor = (status: string) => {
  switch (status) {
    case 'pending': return 'default';
    case 'in_progress': return 'processing';
    case 'error': return 'error';
    case 'review_waiting': return 'warning';
    case 'completed': return 'success';
    default: return 'default';
  }
};
```

3.1.3. 日付表示

フォーマット:

```
new Date(date).toLocaleDateString('ja-JP', {
  year: 'numeric',
  month: '2-digit',
  day: '2-digit',
})
// 例: 2024/01/15
```

nullの場合: `-` を表示

3.1.4. 詳細表示

処理フロー:

1. 詳細ボタンクリック時にonClick()が発火
2. e.stopPropagation()でイベント伝播を停止
3. onViewResult(record)コールバックを実行
4. 親コンポーネントでモーダルやダイアログを表示

3.1.5. 空データ表示

Empty表示:

- image: Empty.PRESENTED_IMAGE_SIMPLE
- description: 多言語対応（「タスクがありません」）

3.2. ボタン・操作の動作

操作	動作	状態変更
カラムヘッダークリック	ソート実行	Tableが管理
詳細ボタンクリック	コールバック実行	親で制御

3.3. バリデーションルール

- なし（表示専用）

---

4. データ仕様

4.1. Props型定義

```
interface TaskResultsTableProps {
  results: TaskResult[];              // タスク結果配列
```

```
  onViewResult: (result: TaskResult) => void; // 詳細表示コールバック
}
```

4.2. TaskResult型定義

```
interface TaskResult {
  workId: string;                  // タスクID（rowKeyとして使用）
  taskName: string;                // タスク名
  methodName: string;              // メソッド名
  assignedWorker: string;          // 割り当てられた作業者
  workStatus: TaskStatus;          // タスクステータス
  progressPercentage: number;      // 進捗率（表示未使用）
  completedAt: string | null;      // 完了日時（表示未使用）
  workCompletedAt: string | null;  // 作業終了日
  reviewCompletedAt: string | null; // レビュー完了日
}
```

4.3. TaskStatus型定義

```
type TaskStatus =
  | 'pending'         // 実行待ち
  | 'in_progress'     // 実行中
  | 'error'           // 異常終了
  | 'review_waiting'  // レビュー待ち
  | 'completed';      // 完了
```

4.4. 入力項目

- results配列（親コンポーネントから渡される）

4.5. 出力項目

- なし（コールバック経由で親に通知）

---

5. API連携

5.1. 使用するAPI一覧

- なし（親コンポーネントがAPIからデータを取得して渡す）

---

6. エラーハンドリング

6.1. エラーケース

エラーケース	処理
results配列が空	Empty表示
日付がnull	`-` 表示
ステータスが未定義	そのまま表示

6.2. エラーメッセージ

- なし（表示専用コンポーネント）

---

7. UI/UXポリシー準拠

7.1. Ant Designコンポーネント使用

コンポーネント	使用箇所	props
Table	テーブル本体	dataSource, columns, rowKey, size, pagination, locale
Button	詳細ボタン	type, size, icon, onClick
Tag	ステータス表示	color
Typography.Text	テキスト表示	strong, style

Empty	空データ表示	description, image

7.2. デザイントークン使用

```
import { theme } from 'antd';
const { token } = theme.useToken();

// 使用箇所
fontSize: token.fontSizeSM  // 日付表示のフォントサイズ
```

7.3. カラーパレット

ステータスTag:

- pending: default（灰色）
- in_progress: processing（青）
- error: error（赤）
- review_waiting: warning（黄）
- completed: success（緑）

7.4. 多言語対応

useLanguage()カスタムフックを使用：

```
const { t } = useLanguage();

t('projectManagement.task_results.task_name')
t('projectManagement.task_results.status_pending')
// など
```

7.5. レスポンシブ対応

- カラム幅: パーセンテージ指定で自動調整
- テーブルサイズ: small（コンパクト表示）

---

8. 実装ファイル

8.1. ソースファイルパス

```
frontend/src/components/project/TaskResultsTable.tsx
```

8.2. 関連コンポーネント

- なし（独立したテーブルコンポーネント）

8.3. カスタムフック

フック	パス	役割
useLanguage	@/contexts/LanguageContext	多言語化

8.4. 型定義

型	パス	役割
TaskStatus	@/types/progress	タスクステータス型

---

9. 既知の課題・TODO

実装待ち

- なし

改善提案

- [ ] 進捗率（progressPercentage）の表示
- [ ] 完了日時（completedAt）の表示

- [ ] 割り当てられた作業者（assignedWorker）の表示
- [ ] フィルタリング機能（ステータス別、メソッド別）
- [ ] 検索機能（タスク名、メソッド名）
- [ ] エクスポート機能（CSV、Excel）
- [ ] 詳細情報のツールチップ表示
- [ ] カラムの表示/非表示切替
- [ ] カラム幅のカスタマイズ
- [ ] 一括選択機能
- [ ] 行クリックで詳細表示

---

10. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-21	初版作成	Cline